

```

--
=====
=====
-- SECTION: ROW LEVEL SECURITY
-- All 96 public tables have RLS ENABLED (none
FORCED).
-- Apply AFTER 05_functions.sql (policies call
hg_is_admin(), etc.)
--
=====
=====

ALTER TABLE public.abandoned_carts ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.academic_levels ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.admin_audit_logs ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.banners ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.batch_orders ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.campuses ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.cart_items ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.catering_requests ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.challenge_completions ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.challenges ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.cron_locks ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.daily_checkins ENABLE ROW LEVEL
SECURITY;

```

```
ALTER TABLE public.delivery_assignments ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.delivery_batches ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.delivery_windows ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.delivery_zones ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.departments ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.device_fingerprints ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.device_tokens ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.event_checkins ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.event_ticket_tiers ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.event_tickets ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.events ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.exclusive_spins ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.feature_flags ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.first_order_gifts ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.flash_redemptions ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.free_side_credits ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.gates ENABLE ROW LEVEL SECURITY;  
ALTER TABLE public.hall_of_fame_inductees ENABLE  
ROW LEVEL SECURITY;  
ALTER TABLE public.hall_of_fame_rewards ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.hostels ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.hp_bundle_purchases ENABLE ROW
```

```
LEVEL SECURITY;
ALTER TABLE public.hp_bundles ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.hp_expiry_log ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.hp_tiers ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.hp_transactions ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.kitchen_settings ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.leaderboard_entries ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.leaderboard_reward_fulfillments
ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.leaderboard_snapshots ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.login_streak_rewards ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.login_streaks ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.marketplace_access_codes ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.marketplace_code_batches ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.marketplace_listings ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.marketplace_purchases ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.marketplace_requests ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.membership_rewards ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.menu_addon_groups ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.menu_addons ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.menu_categories ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.menu_item_variation_groups
```

```
ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.menu_item_variation_options
ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.menu_items ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.milestones ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.monthly_hp_tracker ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.newsletter_subscriptions ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.notification_blasts ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.notification_deliveries ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.notification_log ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.notification_preferences ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.notifications ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.operating_hour_overrides ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.operating_hours ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.order_addon_selections ENABLE
ROW LEVEL SECURITY;
ALTER TABLE public.order_items ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.order_locks ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.order_reviews ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.order_share_events ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.order_status_logs ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.order_streak_rewards ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.order_streaks ENABLE ROW LEVEL
```

```
SECURITY;  
ALTER TABLE public.orders ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.payments ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.profiles ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.promo_code_uses ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.promo_codes ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.push_subscriptions ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.referral_codes ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.referral_milestones ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.referrals ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.reward_redemptions ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.rewards ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.rider_profiles ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.saved_for_later ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.scheduled_notifications ENABLE  
ROW LEVEL SECURITY;  
ALTER TABLE public.squad_members ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.storefront_sections ENABLE ROW  
LEVEL SECURITY;  
ALTER TABLE public.system_settings ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.user_addresses ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.user_milestones ENABLE ROW LEVEL  
SECURITY;  
ALTER TABLE public.user_tiers ENABLE ROW LEVEL
```

```

SECURITY;
ALTER TABLE public.virtual_accounts ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.wallet_topups ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.wallet_transactions ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.wallet_withdrawals ENABLE ROW
LEVEL SECURITY;
ALTER TABLE public.wallets ENABLE ROW LEVEL
SECURITY;
ALTER TABLE public.webhook_events ENABLE ROW LEVEL
SECURITY;

-- ----- POLICIES -----

CREATE POLICY "abandoned_carts: admins all" ON
public.abandoned_carts AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin()) WITH CHECK
(hg_is_admin());
CREATE POLICY "abandoned_carts: users manage own"
ON public.abandoned_carts AS PERMISSIVE FOR ALL TO
authenticated USING ((user_id = ( SELECT auth.uid()
AS uid))) WITH CHECK ((user_id = ( SELECT
auth.uid() AS uid)));
CREATE POLICY academic_levels_public_read ON
public.academic_levels AS PERMISSIVE FOR SELECT TO
public USING ((is_active = true));
CREATE POLICY "admin_audit_logs: admins all" ON
public.admin_audit_logs AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin()) WITH CHECK
(hg_is_admin());
CREATE POLICY "admin_audit_logs: admins read" ON
public.admin_audit_logs AS PERMISSIVE FOR SELECT TO
public USING (hg_is_admin());
CREATE POLICY "banners: admins all" ON
public.banners AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin()) WITH CHECK (hg_is_admin());
CREATE POLICY "banners: public read active" ON
public.banners AS PERMISSIVE FOR SELECT TO public

```

```

USING (((is_active = true) AND ((starts_at IS NULL)
OR (starts_at <= now())) AND ((ends_at IS NULL) OR
(ends_at >= now()))));
CREATE POLICY batch_orders_admin_all ON
public.batch_orders AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin()) WITH CHECK (hg_is_admin());
CREATE POLICY batch_orders_rider_read ON
public.batch_orders AS PERMISSIVE FOR SELECT TO
public USING ((EXISTS ( SELECT 1 FROM
delivery_batches b WHERE ((b.id =
batch_orders.batch_id) AND (b.rider_id = ( SELECT
auth.uid() AS uid))))));
CREATE POLICY "Allow select for everyone" ON
public.campuses AS PERMISSIVE FOR SELECT TO
authenticated, anon USING (true);
CREATE POLICY "cart_items: users manage own" ON
public.cart_items AS PERMISSIVE FOR ALL TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "catering_requests: admins all" ON
public.catering_requests AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "catering_requests: public insert" ON
public.catering_requests AS PERMISSIVE FOR INSERT
TO public WITH CHECK (true);
CREATE POLICY "challenge_completions: admins all"
ON public.challenge_completions AS PERMISSIVE FOR
ALL TO public USING (hg_is_admin());
CREATE POLICY "challenge_completions: users read
own" ON public.challenge_completions AS PERMISSIVE
FOR SELECT TO authenticated USING ((( SELECT
auth.uid() AS uid) = user_id));
CREATE POLICY "challenges: admins all" ON
public.challenges AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "challenges: public read active" ON
public.challenges AS PERMISSIVE FOR SELECT TO
public USING (((is_active = true) OR
hg_is_admin()));

```

```

CREATE POLICY "Admins manage cron_locks" ON
public.cron_locks AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY daily_checkins_own ON
public.daily_checkins AS PERMISSIVE FOR ALL TO
authenticated USING ((user_id = ( SELECT auth.uid()
AS uid))) WITH CHECK ((user_id = ( SELECT
auth.uid() AS uid)));
CREATE POLICY daily_checkins_service ON
public.daily_checkins AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY "delivery_assignments: admins all" ON
public.delivery_assignments AS PERMISSIVE FOR ALL
TO public USING (hg_is_admin());
CREATE POLICY "delivery_assignments: riders read"
ON public.delivery_assignments AS PERMISSIVE FOR
SELECT TO public USING (((rider_id = ( SELECT
auth.uid() AS uid)) OR hg_is_admin()));
CREATE POLICY "delivery_batches: admins all" ON
public.delivery_batches AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "delivery_batches: riders read" ON
public.delivery_batches AS PERMISSIVE FOR SELECT TO
public USING (((rider_id = ( SELECT auth.uid() AS
uid)) OR hg_is_admin()));
CREATE POLICY "delivery_windows: admins all" ON
public.delivery_windows AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "delivery_windows: public read
active" ON public.delivery_windows AS PERMISSIVE
FOR SELECT TO public USING (((is_active = true) OR
hg_is_admin()));
CREATE POLICY "delivery_zones: admins all" ON
public.delivery_zones AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "delivery_zones: public read active"
ON public.delivery_zones AS PERMISSIVE FOR SELECT
TO public USING (((is_active = true) OR
hg_is_admin()));
CREATE POLICY departments_public_read ON

```



```

public.departments AS PERMISSIVE FOR SELECT TO
public USING ((is_active = true));
CREATE POLICY "Admins manage device fingerprints"
ON public.device_fingerprints AS PERMISSIVE FOR ALL
TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Users manage own device tokens" ON
public.device_tokens AS PERMISSIVE FOR ALL TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "event_checkins: admins all" ON
public.event_checkins AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY event_tiers_admin ON
public.event_ticket_tiers AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY event_tiers_read_all ON
public.event_ticket_tiers AS PERMISSIVE FOR SELECT
TO authenticated, anon USING ((is_active = true));
CREATE POLICY event_tiers_service ON
public.event_ticket_tiers AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY "event_tickets: admins all" ON
public.event_tickets AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "event_tickets: users read own" ON
public.event_tickets AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "events: admins all" ON public.events
AS PERMISSIVE FOR ALL TO public USING
(hg_is_admin());
CREATE POLICY "events: public read published" ON
public.events AS PERMISSIVE FOR SELECT TO public

```

```

USING (((is_published = true) OR hg_is_admin())));
CREATE POLICY "Allow authenticated users to read
their own exclusive spins" ON
public.exclusive_spins AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY exclusive_spins_admin ON
public.exclusive_spins AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY exclusive_spins_own ON
public.exclusive_spins AS PERMISSIVE FOR SELECT TO
authenticated USING ((user_id = ( SELECT auth.uid()
AS uid)));
CREATE POLICY exclusive_spins_service ON
public.exclusive_spins AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY feature_flags_admin_write ON
public.feature_flags AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY feature_flags_read_all ON
public.feature_flags AS PERMISSIVE FOR SELECT TO
authenticated, anon USING (true);
CREATE POLICY feature_flags_service_all ON
public.feature_flags AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY "Admins manage first_order_gifts" ON
public.first_order_gifts AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Users view own gift" ON
public.first_order_gifts AS PERMISSIVE FOR SELECT
TO authenticated USING ((( SELECT auth.uid() AS

```

```

uid) = user_id));
CREATE POLICY "Admins manage flash_redemptions" ON
public.flash_redemptions AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "Anyone can view flash_redemptions"
ON public.flash_redemptions AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY free_side_credits_admin ON
public.free_side_credits AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY free_side_credits_own ON
public.free_side_credits AS PERMISSIVE FOR SELECT
TO authenticated USING ((( SELECT auth.uid() AS
uid) = user_id));
CREATE POLICY free_side_credits_service ON
public.free_side_credits AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY anon_read_gates ON public.gates AS
PERMISSIVE FOR SELECT TO authenticated, anon USING
((is_active = true));
CREATE POLICY service_role_all_gates ON
public.gates AS PERMISSIVE FOR ALL TO service_role
USING (true) WITH CHECK (true);
CREATE POLICY "Admins manage hall_of_fame" ON
public.hall_of_fame_inductees AS PERMISSIVE FOR ALL
TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Anyone reads hall_of_fame" ON
public.hall_of_fame_inductees AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY hof_rewards_admin ON
public.hall_of_fame_rewards AS PERMISSIVE FOR ALL
TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =

```

```

'admin'::user_role))));
CREATE POLICY hof_rewards_own_read ON
public.hall_of_fame_rewards AS PERMISSIVE FOR
SELECT TO authenticated USING ((user_id = ( SELECT
auth.uid() AS uid)));
CREATE POLICY hof_rewards_service ON
public.hall_of_fame_rewards AS PERMISSIVE FOR ALL
TO service_role USING (true) WITH CHECK (true);
CREATE POLICY anon_read_hostels ON public.hostels
AS PERMISSIVE FOR SELECT TO authenticated, anon
USING ((is_active = true));
CREATE POLICY service_role_all_hostels ON
public.hostels AS PERMISSIVE FOR ALL TO
service_role USING (true) WITH CHECK (true);
CREATE POLICY "Admins manage hp_bundle_purchases"
ON public.hp_bundle_purchases AS PERMISSIVE FOR ALL
TO public USING (hg_is_admin());
CREATE POLICY "Users view own hp_bundle_purchases"
ON public.hp_bundle_purchases AS PERMISSIVE FOR
SELECT TO authenticated USING ((( SELECT auth.uid()
AS uid) = event_host_id));
CREATE POLICY "Admins manage hp_bundles" ON
public.hp_bundles AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "Anyone can view active hp_bundles"
ON public.hp_bundles AS PERMISSIVE FOR SELECT TO
public USING ((is_active = true));
CREATE POLICY hp_expiry_log_admin_all ON
public.hp_expiry_log AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin()) WITH CHECK
(hg_is_admin());
CREATE POLICY hp_expiry_log_users_read_own ON
public.hp_expiry_log AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "hp_tiers: admins all" ON
public.hp_tiers AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "hp_tiers: public read active" ON
public.hp_tiers AS PERMISSIVE FOR SELECT TO public

```

```

USING (((is_active = true) OR hg_is_admin()));
CREATE POLICY "hp_transactions: admins all" ON
public.hp_transactions AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "hp_transactions: users read own" ON
public.hp_transactions AS PERMISSIVE FOR SELECT TO
public USING (((user_id = ( SELECT auth.uid() AS
uid)) OR hg_is_admin()));
CREATE POLICY "Anyone can view kitchen_settings" ON
public.kitchen_settings AS PERMISSIVE FOR SELECT TO
public USING (true);
CREATE POLICY "Kitchen and admins manage
kitchen_settings" ON public.kitchen_settings AS
PERMISSIVE FOR ALL TO public USING ((hg_is_admin()
OR (hg_current_user_role() =
'kitchen':user_role)));
CREATE POLICY "leaderboard_entries: admins all" ON
public.leaderboard_entries AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "leaderboard_entries: public read" ON
public.leaderboard_entries AS PERMISSIVE FOR SELECT
TO public USING (true);
CREATE POLICY lb_fulfill_admin ON
public.leaderboard_reward_fulfillments AS
PERMISSIVE FOR ALL TO authenticated USING ((EXISTS
( SELECT 1 FROM profiles WHERE ((profiles.id = (
SELECT auth.uid() AS uid)) AND (profiles.role =
'admin':user_role))));
CREATE POLICY lb_fulfill_own_read ON
public.leaderboard_reward_fulfillments AS
PERMISSIVE FOR SELECT TO authenticated USING (((
SELECT auth.uid() AS uid) = user_id));
CREATE POLICY lb_fulfill_service ON
public.leaderboard_reward_fulfillments AS
PERMISSIVE FOR ALL TO service_role USING (true)
WITH CHECK (true);
CREATE POLICY "leaderboard_snapshots: admins all"
ON public.leaderboard_snapshots AS PERMISSIVE FOR
ALL TO public USING (hg_is_admin());
CREATE POLICY "leaderboard_snapshots: public read"

```

```

ON public.leaderboard_snapshots AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY "Admins manage login_streak_rewards"
ON public.login_streak_rewards AS PERMISSIVE FOR
ALL TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Anyone reads login_streak_rewards"
ON public.login_streak_rewards AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY "Admins manage streaks" ON
public.login_streaks AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Users view own streak" ON
public.login_streaks AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "marketplace_access_codes: admins
all" ON public.marketplace_access_codes AS
PERMISSIVE FOR ALL TO public USING (hg_is_admin());
CREATE POLICY "marketplace_code_batches: admins
all" ON public.marketplace_code_batches AS
PERMISSIVE FOR ALL TO public USING (hg_is_admin());
CREATE POLICY "marketplace_listings: admins all" ON
public.marketplace_listings AS PERMISSIVE FOR ALL
TO public USING (hg_is_admin());
CREATE POLICY "marketplace_listings: public read
active" ON public.marketplace_listings AS
PERMISSIVE FOR SELECT TO public USING (((status =
'active'::text) AND (COALESCE(is_out_of_stock,
false) = false)) OR hg_is_admin());
CREATE POLICY "marketplace_purchases: admins all"
ON public.marketplace_purchases AS PERMISSIVE FOR
ALL TO public USING (hg_is_admin());
CREATE POLICY "marketplace_purchases: users read
own" ON public.marketplace_purchases AS PERMISSIVE

```

```

FOR SELECT TO authenticated USING ((( SELECT
auth.uid() AS uid) = user_id));
CREATE POLICY "Admins manage marketplace requests"
ON public.marketplace_requests AS PERMISSIVE FOR
ALL TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Admins manage membership_rewards" ON
public.membership_rewards AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Anyone reads membership_rewards" ON
public.membership_rewards AS PERMISSIVE FOR SELECT
TO public USING (true);
CREATE POLICY "Admins manage addon groups" ON
public.menu_addon_groups AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Anyone can view addon groups" ON
public.menu_addon_groups AS PERMISSIVE FOR SELECT
TO public USING (true);
CREATE POLICY "Admins manage menu_addons" ON
public.menu_addons AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "Anyone can view available
menu_addons" ON public.menu_addons AS PERMISSIVE
FOR SELECT TO public USING (((is_available = true)
AND (is_archived = false)));
CREATE POLICY "menu_categories: admins all" ON
public.menu_categories AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "menu_categories: public read active"
ON public.menu_categories AS PERMISSIVE FOR SELECT
TO public USING (((is_active = true) OR
hg_is_admin()));

```

```

CREATE POLICY "Admins manage
menu_item_variation_groups" ON
public.menu_item_variation_groups AS PERMISSIVE FOR
ALL TO public USING (hg_is_admin());
CREATE POLICY "Anyone can view
menu_item_variation_groups" ON
public.menu_item_variation_groups AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY "Admins manage
menu_item_variation_options" ON
public.menu_item_variation_options AS PERMISSIVE
FOR ALL TO public USING (hg_is_admin());
CREATE POLICY "Anyone can view available
variation_options" ON
public.menu_item_variation_options AS PERMISSIVE
FOR SELECT TO public USING ((is_available = true));
CREATE POLICY "menu_items: admins all" ON
public.menu_items AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "menu_items: public read available"
ON public.menu_items AS PERMISSIVE FOR SELECT TO
public USING (((is_available = true) AND
(deleted_at IS NULL)) OR hg_is_admin());
CREATE POLICY "Admins manage milestones" ON
public.milestones AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Anyone can view active milestones"
ON public.milestones AS PERMISSIVE FOR SELECT TO
public USING ((is_active = true));
CREATE POLICY "Admins manage monthly trackers" ON
public.monthly_hp_tracker AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Users view own monthly tracker" ON
public.monthly_hp_tracker AS PERMISSIVE FOR SELECT

```



```

TO public USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "newsletter_subscriptions: admins
all" ON public.newsletter_subscriptions AS
PERMISSIVE FOR ALL TO public USING (hg_is_admin());
CREATE POLICY "newsletter_subscriptions: public
insert" ON public.newsletter_subscriptions AS
PERMISSIVE FOR INSERT TO public WITH CHECK (true);
CREATE POLICY "notification_blasts: admins all" ON
public.notification_blasts AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin()) WITH CHECK
(hg_is_admin());
CREATE POLICY "notification_deliveries: admins all"
ON public.notification_deliveries AS PERMISSIVE FOR
ALL TO public USING (hg_is_admin());
CREATE POLICY "notification_deliveries: users read"
ON public.notification_deliveries AS PERMISSIVE FOR
SELECT TO authenticated USING ((user_id = ( SELECT
auth.uid() AS uid)));
CREATE POLICY "Admins manage notification_log" ON
public.notification_log AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Users manage own notification
preferences" ON public.notification_preferences AS
PERMISSIVE FOR ALL TO authenticated USING (((
SELECT auth.uid() AS uid) = user_id)) WITH CHECK
((( SELECT auth.uid() AS uid) = user_id));
CREATE POLICY "notifications: admins all" ON
public.notifications AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "notifications: users read own" ON
public.notifications AS PERMISSIVE FOR SELECT TO
authenticated USING (((user_id = ( SELECT
auth.uid() AS uid)) OR hg_is_admin()));
CREATE POLICY "notifications: users update own" ON
public.notifications AS PERMISSIVE FOR UPDATE TO
authenticated USING ((user_id = ( SELECT auth.uid()

```

```

AS uid))) WITH CHECK ((user_id = ( SELECT
auth.uid() AS uid)));
CREATE POLICY "operating_hour_overrides: admins
all" ON public.operating_hour_overrides AS
PERMISSIVE FOR ALL TO public USING (hg_is_admin())
WITH CHECK (hg_is_admin());
CREATE POLICY "operating_hour_overrides: public
read" ON public.operating_hour_overrides AS
PERMISSIVE FOR SELECT TO public USING (true);
CREATE POLICY "operating_hours: admins all" ON
public.operating_hours AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "operating_hours: public read" ON
public.operating_hours AS PERMISSIVE FOR SELECT TO
public USING (true);
CREATE POLICY "Admins/kitchen manage addon
selections" ON public.order_addon_selections AS
PERMISSIVE FOR ALL TO authenticated USING ((EXISTS
( SELECT 1 FROM profiles WHERE ((profiles.id = (
SELECT auth.uid() AS uid)) AND (profiles.role = ANY
(ARRAY['admin'::user_role,
'kitchen'::user_role]))))););
CREATE POLICY "order_items: admins all" ON
public.order_items AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "order_items: users read own" ON
public.order_items AS PERMISSIVE FOR SELECT TO
public USING ((EXISTS ( SELECT 1 FROM orders o
WHERE ((o.id = order_items.order_id) AND
((o.user_id = ( SELECT auth.uid() AS uid)) OR
hg_is_admin()))))););
CREATE POLICY "Admins manage all locks" ON
public.order_locks AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role)))));
CREATE POLICY "Users manage own locks" ON
public.order_locks AS PERMISSIVE FOR ALL TO
authenticated USING ((( SELECT auth.uid() AS uid) =

```

```

user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "order_reviews: admins all" ON
public.order_reviews AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "order_reviews: public read" ON
public.order_reviews AS PERMISSIVE FOR SELECT TO
public USING (true);
CREATE POLICY "order_reviews: users update" ON
public.order_reviews AS PERMISSIVE FOR UPDATE TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY order_reviews_one_per_order_insert ON
public.order_reviews AS PERMISSIVE FOR INSERT TO
authenticated WITH CHECK ((((( SELECT auth.uid() AS
uid) = user_id) AND (EXISTS ( SELECT 1 FROM orders
o WHERE ((o.id = order_reviews.order_id) AND
(o.user_id = ( SELECT auth.uid() AS uid)) AND
(o.status = 'delivered':order_status)))) AND (NOT
(EXISTS ( SELECT 1 FROM order_reviews er WHERE
(er.order_id = order_reviews.order_id))))));
CREATE POLICY "Admins manage share events" ON
public.order_share_events AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin':user_role)))));
CREATE POLICY "Users view own share events" ON
public.order_share_events AS PERMISSIVE FOR SELECT
TO authenticated USING ((( SELECT auth.uid() AS
uid) = user_id));
CREATE POLICY "order_status_logs: admins all" ON
public.order_status_logs AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "order_status_logs: users read own"
ON public.order_status_logs AS PERMISSIVE FOR
SELECT TO public USING ((EXISTS ( SELECT 1 FROM
orders WHERE ((orders.id =
order_status_logs.order_id) AND (orders.user_id = (
SELECT auth.uid() AS uid))))));

```

```

CREATE POLICY "Admins manage order_streak_rewards"
ON public.order_streak_rewards AS PERMISSIVE FOR
ALL TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Anyone reads order_streak_rewards"
ON public.order_streak_rewards AS PERMISSIVE FOR
SELECT TO public USING (true);
CREATE POLICY "Admins manage order_streaks" ON
public.order_streaks AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Users view own order streak" ON
public.order_streaks AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "orders: admins delete" ON
public.orders AS PERMISSIVE FOR DELETE TO public
USING (hg_is_admin());
CREATE POLICY "orders: admins update all" ON
public.orders AS PERMISSIVE FOR UPDATE TO public
USING (hg_is_admin());
CREATE POLICY "orders: users insert own" ON
public.orders AS PERMISSIVE FOR INSERT TO public
WITH CHECK (((user_id = ( SELECT auth.uid() AS
uid)) OR (user_id IS NULL)));
CREATE POLICY "orders: users read own" ON
public.orders AS PERMISSIVE FOR SELECT TO public
USING (((user_id = ( SELECT auth.uid() AS uid)) OR
(hg_current_user_role() = ANY
(ARRAY['kitchen'::user_role, 'rider'::user_role,
'admin'::user_role, 'super_admin'::user_role]))));
CREATE POLICY orders_admin_all ON public.orders AS
PERMISSIVE FOR ALL TO public USING (hg_is_admin())
WITH CHECK (hg_is_admin());
CREATE POLICY "payments: admins all" ON
public.payments AS PERMISSIVE FOR ALL TO public

```

```

USING (hg_is_admin());
CREATE POLICY "payments: users read own" ON
public.payments AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "profiles: admins delete" ON
public.profiles AS PERMISSIVE FOR DELETE TO public
USING (hg_is_admin());
CREATE POLICY "profiles: users read own" ON
public.profiles AS PERMISSIVE FOR SELECT TO public
USING (((id = ( SELECT auth.uid() AS uid)) OR
hg_is_admin()));
CREATE POLICY "profiles: users update own" ON
public.profiles AS PERMISSIVE FOR UPDATE TO
authenticated USING ((id = ( SELECT auth.uid() AS
uid))) WITH CHECK (((id = ( SELECT auth.uid() AS
uid)) AND (role = ( SELECT p.role FROM profiles p
WHERE (p.id = ( SELECT auth.uid() AS uid)))) AND
(hp_balance = ( SELECT p.hp_balance FROM profiles p
WHERE (p.id = ( SELECT auth.uid() AS uid)))) AND
(wallet_balance = ( SELECT p.wallet_balance FROM
profiles p WHERE (p.id = ( SELECT auth.uid() AS
uid))))));
CREATE POLICY profiles_admin_all ON public.profiles
AS PERMISSIVE FOR ALL TO public USING
(hg_is_admin()) WITH CHECK (hg_is_admin());
CREATE POLICY "promo_code_uses: admins all" ON
public.promo_code_uses AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "promo_code_uses: users read own" ON
public.promo_code_uses AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "promo_codes: admins all" ON
public.promo_codes AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin()) WITH CHECK (hg_is_admin());
CREATE POLICY "promo_codes: authenticated read
active" ON public.promo_codes AS PERMISSIVE FOR
SELECT TO public USING (((is_active = true) AND ((
SELECT auth.uid() AS uid) IS NOT NULL)) OR

```

```

hg_is_admin());
CREATE POLICY "push_subscriptions: admins read all"
ON public.push_subscriptions AS PERMISSIVE FOR
SELECT TO public USING (hg_is_admin());
CREATE POLICY "push_subscriptions: users crud own"
ON public.push_subscriptions AS PERMISSIVE FOR ALL
TO authenticated USING ((user_id = ( SELECT
auth.uid() AS uid))) WITH CHECK ((user_id = (
SELECT auth.uid() AS uid)));
CREATE POLICY "referral_codes: admins all" ON
public.referral_codes AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "referral_codes: users read own" ON
public.referral_codes AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "Admins manage referral_milestones"
ON public.referral_milestones AS PERMISSIVE FOR ALL
TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))));
CREATE POLICY "Anyone reads referral_milestones" ON
public.referral_milestones AS PERMISSIVE FOR SELECT
TO public USING (true);
CREATE POLICY "referrals: admins all" ON
public.referrals AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "referrals: users read own" ON
public.referrals AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid)
= referrer_id) OR (( SELECT auth.uid() AS uid) =
referred_user_id));
CREATE POLICY "reward_redemptions: admins all" ON
public.reward_redemptions AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "reward_redemptions: users read own"
ON public.reward_redemptions AS PERMISSIVE FOR
SELECT TO authenticated USING ((( SELECT auth.uid()
AS uid) = user_id));

```

```

CREATE POLICY "rewards: admins all" ON
public.rewards AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "rewards: public read active" ON
public.rewards AS PERMISSIVE FOR SELECT TO public
USING (((is_active = true) OR hg_is_admin()));
CREATE POLICY "Admins manage all rider profiles" ON
public.rider_profiles AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Riders manage own profile" ON
public.rider_profiles AS PERMISSIVE FOR ALL TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "Users manage own saved items" ON
public.saved_for_later AS PERMISSIVE FOR ALL TO
public USING ((( SELECT auth.uid() AS uid) =
user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "Admins manage
scheduled_notifications" ON
public.scheduled_notifications AS PERMISSIVE FOR
ALL TO authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Admins manage squad members" ON
public.squad_members AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))))));
CREATE POLICY "Users view own squad entries" ON
public.squad_members AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "storefront_sections: admins all" ON

```

```

public.storefront_sections AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "storefront_sections: public read
active" ON public.storefront_sections AS PERMISSIVE
FOR SELECT TO public USING (((is_active = true) OR
hg_is_admin()));
CREATE POLICY "Admins manage system_settings" ON
public.system_settings AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))));
CREATE POLICY "Anyone can read system_settings" ON
public.system_settings AS PERMISSIVE FOR SELECT TO
public USING ((is_public = true));
CREATE POLICY "system_settings: admins all" ON
public.system_settings AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "system_settings: public read
nonsensitive" ON public.system_settings AS
PERMISSIVE FOR SELECT TO public USING (((is_public
= true) OR hg_is_admin()));
CREATE POLICY "user_addresses: admins all" ON
public.user_addresses AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin()) WITH CHECK
(hg_is_admin());
CREATE POLICY "user_addresses: users crud own" ON
public.user_addresses AS PERMISSIVE FOR ALL TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id)) WITH CHECK ((( SELECT auth.uid() AS uid)
= user_id));
CREATE POLICY "Admins manage user_milestones" ON
public.user_milestones AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin'::user_role))));
CREATE POLICY "Users view own milestones" ON
public.user_milestones AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =

```



```

user_id));
CREATE POLICY "user_tiers: admins all" ON
public.user_tiers AS PERMISSIVE FOR ALL TO public
USING (hg_is_admin());
CREATE POLICY "user_tiers: users read own" ON
public.user_tiers AS PERMISSIVE FOR SELECT TO
public USING ((user_id = ( SELECT auth.uid() AS
uid)));
CREATE POLICY "virtual_accounts: admins all" ON
public.virtual_accounts AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "virtual_accounts: users read own" ON
public.virtual_accounts AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "wallet_topups: admins all" ON
public.wallet_topups AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "wallet_topups: users insert own" ON
public.wallet_topups AS PERMISSIVE FOR INSERT TO
authenticated WITH CHECK ((( SELECT auth.uid() AS
uid) = user_id));
CREATE POLICY "wallet_topups: users read own" ON
public.wallet_topups AS PERMISSIVE FOR SELECT TO
authenticated USING ((( SELECT auth.uid() AS uid) =
user_id));
CREATE POLICY "wallet_transactions: admins all" ON
public.wallet_transactions AS PERMISSIVE FOR ALL TO
public USING (hg_is_admin());
CREATE POLICY "wallet_transactions: users read own"
ON public.wallet_transactions AS PERMISSIVE FOR
SELECT TO authenticated USING ((( SELECT auth.uid()
AS uid) = user_id));
CREATE POLICY "Admins manage all withdrawals" ON
public.wallet_withdrawals AS PERMISSIVE FOR ALL TO
authenticated USING ((EXISTS ( SELECT 1 FROM
profiles WHERE ((profiles.id = ( SELECT auth.uid()
AS uid)) AND (profiles.role =
'admin':user_role)))));
CREATE POLICY "Users view own withdrawals" ON

```

```
public.wallet_withdrawals AS PERMISSIVE FOR SELECT  
TO authenticated USING ((( SELECT auth.uid() AS  
uid) = user_id));  
CREATE POLICY "wallets: admins all" ON  
public.wallets AS PERMISSIVE FOR ALL TO public  
USING (hg_is_admin());  
CREATE POLICY "wallets: users read own" ON  
public.wallets AS PERMISSIVE FOR SELECT TO  
authenticated USING ((( SELECT auth.uid() AS uid) =  
user_id));  
CREATE POLICY "Admins manage webhook_events" ON  
public.webhook_events AS PERMISSIVE FOR ALL TO  
public USING (hg_is_admin());
```